Fawaterak Payment Gateway Integration Guide

ASP.NET Core Web API — Full Implementation Reference

Table of Contents

1. What is Fawaterak?
2. How the Payment Flow Works
3. Project Architecture Overview
4. Step-by-Step Implementation
5. Configuration & Settings
6. The Request/Response Models
7. The Service Layer
8. The Controller Layer
9. Webhook & Callback Handling
10. Common Errors & Fixes
11. Testing Checklist

---

1. What is Fawaterak?

Fawaterak is an Egyptian payment aggregator that supports multiple payment methods through a single API — including Visa/Mastercard, Fawry, Vodafone Cash, Orange Cash, and installment plans.

Instead of integrating with each payment provider separately, Fawaterak provides one unified API endpoint. You create an "invoice link" through their API, redirect your user to the hosted payment page, and Fawaterak handles the actual payment processing. Once done, they call your callback URL with the result.

Key concepts:

- `providerKey` — identifies your specific payment provider configuration (e.g., which bank or wallet you want to use). **This is required in every request body.**
- `apiKey` — your authentication token, sent as a Bearer token in the Authorization header.
- Invoice Link — a hosted payment page URL that Fawaterak generates for each transaction.
- Callback URL — your endpoint that Fawaterak calls (via webhook) when a payment is completed.

---

2. How the Payment Flow Works

```
┌──────────────────┐        ┌──────────────────────────┐        ┌──────────
│   Client    │        │  Your ASP.NET    │        │  Fawaterak API      │
│  (Browser/  │        │  Core Web API    │        │  (staging.fawaterak  │
│   Mobile)   │        │                  │        │  .com)              │
└──────────┬──────────┘        └──────────────┬──────────────────────┘        └──────────
     │                    │                          │
     │  POST /api/payment      │                          │
     │  /checkout              │                          │
     │ (basket + user info)    │                          │
     │──────────────────────────────────────────>│                          │
     │                    │  POST /api/v2/              │
     │                    │  createInvoiceLink          │
     │                    │  {providerKey, cartTotal,    │
     │                    │   customer, cartItems,       │
     │                    │   redirectionUrls}           │
     │                    │──────────────────────────────────────────────>
     │                    │                          │
     │                    │   200 OK                    │
     │                    │   {status:"success",         │
     │                    │    data:{url, invoiceId}}    │
     │                    │<──────────────────────────────────────────────
     │                    │                          │
     │  {paymentUrl,           │                          │
     │   invoiceId}            │                          │
     │<──────────────────────────────────────────│                          │
```

```
    |                    |                        |
    |  User visits paymentUrl |                        |
    | (hosted by Fawaterak)   |                        |
    |──────────────────────────────────────────────────────────────────────
    |                    |                        |
    |                    |  (User pays on Fawaterak)    |
    |                    |                        |
    |                    |  POST /api/payment/callback  |
    |                    |  (webhook with payment       |
    |                    |   result)                    |
    |                    |<──────────────────────────────────────────
    |                    |                        |
    |  Redirect to           |                        |
    |  successUrl / failUrl   |                        |
    |<──────────────────────────────────────|                        |
```

Step-by-step in plain English:

1. Your frontend calls your API's `/api/payment/checkout` endpoint with the basket and customer info.
2. Your API builds a `createInvoiceLink` request and sends it to Fawaterak.
3. Fawaterak returns a payment URL (hosted page) and an invoice ID.
4. Your API returns this URL to the client.
5. The client opens the Fawaterak payment page in the browser.
6. The user completes payment on Fawaterak's hosted page.
7. Fawaterak calls your `callbackUrl` (webhook) with the result.
8. Fawaterak redirects the user to your `successUrl` or `failUrl`.

---

3. Project Architecture Overview

```
ECommerceSolution/
 |
 ├── ECommerce.Domain/            ← Entities (Order, Product, etc.)
 |
 ├── ECommerce.Persistence/       ← DbContext, Repositories, EF Core
```

```
│
├── ECommerce.SharedLibirary/      ← Shared DTOs, Settings, Common Resu
│   ├── DTO's/
│   │   └── PaymentDTOs/
│   │       ├── CheckoutDTO.cs         ← Input from the client
│   │       ├── CreateInvoiceDTO.cs    ← Internal DTO for Fawaterak service
│   │       └── FawaterakCallbackPayload.cs ← Webhook payload shape
│   └── Settings/
│       └── FwaterkSettings.cs         ← Strongly-typed config
│
├── ECommerce.Services.Abstraction/  ← Interfaces (IFawaterakService, etc.
│
├── ECommerce.Service/             ← Business logic implementations
│   └── Servicies/
│       └── Payment Service/
│           ├── FawaterakModule.cs      ← Request/Response model classes
│           └── FawaterakService.cs     ← HTTP calls to Fawaterak
│
├── ECommerce.Presentation/        ← Controllers
│
└── ECommerce.Web/                 ← Entry point, DI registration, appsettings
    ├── Program.cs
    └── appsettings.json
```

Why this separation matters:

Layer	Responsibility	What it knows about Fawaterak
Domain	Business entities	Nothing
SharedLibrary	Shared contracts (DTOs, settings)	Settings shape + public DTOs
Service.Abstraction	Interfaces	Only the interface contract
Service	Actual implementation	All of it — HTTP calls, JSON mapping
Presentation	HTTP endpoints	Only the service interface
Web	Wiring everything together	Reads config, registers services

This is a clean layered architecture. The controller never touches `HttpClient` directly, and the domain never knows a payment gateway exists.

---

4. Step-by-Step Implementation

Step 1 — Define the Settings Class

Create a strongly-typed settings class in `ECommerce.SharedLibirary/Settings/FwaterkSettings.cs`:

```
namespace ECommerce.SharedLibirary.Settings
{
    public class FawaterakSettings
    {
        public string ApiKey { get; set; } = default!;
        public string ProviderKey { get; set; } = default!;
        public string BaseUrl { get; set; } = default!;
        public string SuccessUrl { get; set; } = default!;
        public string FailUrl { get; set; } = default!;
        public string PendingUrl { get; set; } = default!;
```

```
        public string CallbackUrl { get; set; } = default!;
    }
}
```

What each field does:

Field	Description
`ApiKey`	Bearer token for Authorization header
`ProviderKey`	Identifies payment provider (e.g., "FAWATERAK.2409") — **required in the request body**
`BaseUrl`	Fawaterak base URL (`https://staging.fawaterak.com` for testing, `https://app.fawaterak.com` for production)
`SuccessUrl`	Where Fawaterak redirects the user after successful payment
`FailUrl`	Where Fawaterak redirects the user after failed payment
`PendingUrl`	Where Fawaterak redirects if payment is pending
`CallbackUrl`	Your webhook endpoint — Fawaterak POSTs payment result here

---

Step 2 — Add Settings to appsettings.json

```
{
  "FawaterakSettings": {
    "ApiKey": "YOUR_API_KEY_FROM_FAWATERAK_DASHBOARD",
    "ProviderKey": "YOUR_PROVIDER_KEY",
    "BaseUrl": "https://staging.fawaterak.com",
    "SuccessUrl": "https://YOUR_DOMAIN/api/payment/success",
    "FailUrl": "https://YOUR_DOMAIN/api/payment/fail",
    "PendingUrl": "https://YOUR_DOMAIN/api/payment/pending",
    "CallbackUrl": "https://YOUR_DOMAIN/api/payment/callback"
```

```
  }
}
```

> **Important for local development:** The `SuccessUrl`, `FailUrl`, `PendingUrl`, and `CallbackUrl` must be publicly accessible URLs — Fawaterak's servers call them from outside your machine. Use `ngrok` to expose your local port:
>
> ```
> ngrok http 5262
> ```
>
> Then replace `YOUR_DOMAIN` with your ngrok URL (e.g., https://abc123.ngrok-free.app). **Restart ngrok** if it has expired and update these 4 values.

Step 3 — Define the DTOs

Public-facing DTO (what the client sends to your API)

ECommerce.SharedLibirary/DTO's/PaymentDTOs/CreateInvoiceDTO.cs:

```
namespace ECommerce.SharedLibirary.DTO_s.PaymentDTOs
{
    public class CreateInvoiceDTO
    {
        public string CartTotal { get; set; } = default!;  // "150.00"
        public string FirstName { get; set; } = default!;
        public string LastName { get; set; } = default!;
        public string Email { get; set; } = default!;
        public string Phone { get; set; } = default!;
        public string Address { get; set; } = default!;
        public List<CreateInvoiceItemDTO> Items { get; set; } = [];
    }

    public class CreateInvoiceItemDTO
    {
        public string Name { get; set; } = default!;
        public string Price { get; set; } = default!;    // "50.00"
```

```
        public string Quantity { get; set; } = default!; // "3"
    }
}
```

> **Why strings for Price and CartTotal?** Fawaterak's API expects numeric values as strings (e.g., `"150.00"` not `150.00`). Keeping the DTO as strings avoids any conversion issues.

Step 4 — Define the Fawaterak Request/Response Models

`ECommerce.Service/Servicies/Payment Service/FawaterakModule.cs`:

```
using System.Text.Json.Serialization;

namespace ECommerce.Services.Servicies.Payment_Service
{
    internal class FawaterakInvoiceRequest
    {
        // ✅ REQUIRED — identifies your payment provider configuration
        [JsonPropertyName("providerKey")]
        public string ProviderKey { get; set; } = default!;

        [JsonPropertyName("cartTotal")]
        public string CartTotal { get; set; } = default!;

        [JsonPropertyName("currency")]
        public string Currency { get; set; } = "EGP";

        [JsonPropertyName("customer")]
        public FawaterakCustomer Customer { get; set; } = default!;

        [JsonPropertyName("cartItems")]
        public List<FawaterakCartItem> CartItems { get; set; } = [];

        [JsonPropertyName("redirectionUrls")]
        public FawaterakRedirectionUrls RedirectionUrls { get; set; } = default!;
```

```
}

internal class FawaterakCustomer
{
    [JsonPropertyName("first_name")]
    public string FirstName { get; set; } = default!;

    [JsonPropertyName("last_name")]
    public string LastName { get; set; } = default!;

    [JsonPropertyName("email")]
    public string Email { get; set; } = default!;

    [JsonPropertyName("phone")]
    public string Phone { get; set; } = default!;

    [JsonPropertyName("address")]
    public string Address { get; set; } = default!;
}

internal class FawaterakCartItem
{
    [JsonPropertyName("name")]
    public string Name { get; set; } = default!;

    [JsonPropertyName("price")]
    public string Price { get; set; } = default!;

    [JsonPropertyName("quantity")]
    public string Quantity { get; set; } = default!;
}

internal class FawaterakRedirectionUrls
{
    [JsonPropertyName("successUrl")]
    public string SuccessUrl { get; set; } = default!;

    [JsonPropertyName("failUrl")]
```

```
        public string FailUrl { get; set; } = default!;

        [JsonPropertyName("pendingUrl")]
        public string PendingUrl { get; set; } = default!;

        [JsonPropertyName("callbackUrl")]
        public string CallbackUrl { get; set; } = default!;
    }

    // Response models
    internal class FawaterakInvoiceResponse
    {
        [JsonPropertyName("status")]
        public string Status { get; set; } = default!;  // "success" or "error"

        [JsonPropertyName("data")]
        public FawaterakInvoiceData Data { get; set; } = default!;
    }

    internal class FawaterakInvoiceData
    {
        [JsonPropertyName("url")]
        public string Url { get; set; } = default!;  // Payment page URL

        [JsonPropertyName("invoiceId")]
        public long InvoiceId { get; set; }
    }
}
```

Key JSON naming rules — why `[JsonPropertyName]` is needed:

`System.Text.Json` by default uses the C# property name as-is. But Fawaterak's API requires specific casing:

C# Property	Fawaterak expects	JsonPropertyName
FirstName	first_name	[JsonPropertyName("first_name")]
ProviderKey	providerKey	[JsonPropertyName("providerKey")]
CartTotal	cartTotal	[JsonPropertyName("cartTotal")]
SuccessUrl	successUrl	[JsonPropertyName("successUrl")]

Without these attributes, the JSON would be sent with wrong field names and Fawaterak would reject the request.

Step 5 — Define the Service Interface

ECommerce.Services.Abstraction/IFawaterakService.cs:

```
using ECommerce.SharedLibirary.DTO_s.PaymentDTOs;

namespace ECommerce.Services.Abstraction
{
    public interface IFawaterakService
    {
        Task<(string PaymentUrl, string InvoiceId)> CreateInvoiceAsync(CreateI
    }
}
```

The interface returns a **value tuple** (string PaymentUrl, string InvoiceId) — a clean way to return two related values without creating a wrapper class.

Step 6 — Implement the Service

ECommerce.Service/Servicies/Payment Service/FawaterakService.cs:

```
using ECommerce.Services.Abstraction;
using ECommerce.SharedLibirary.DTO_s.PaymentDTOs;
using ECommerce.SharedLibirary.Settings;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;

namespace ECommerce.Services.Servicies.Payment_Service;

public class FawaterakService : IFawaterakService
{
    private readonly HttpClient _httpClient;
    private readonly FawaterakSettings _settings;
    private readonly ILogger<FawaterakService> _logger;

    // Separate options for serializing (sending to Fawaterak)
    // vs deserializing (reading Fawaterak's response)
    private static readonly JsonSerializerOptions _serializeOptions = new()
    {
        PropertyNamingPolicy = null,    // Use [JsonPropertyName] attributes exa
        WriteIndented = false
    };

    private static readonly JsonSerializerOptions _deserializeOptions = new(
    {
        PropertyNameCaseInsensitive = true  // Be flexible when reading response
    };

    public FawaterakService(
        HttpClient httpClient,
        IOptions<FawaterakSettings> settings,
        ILogger<FawaterakService> logger)
```

```
{
    _settings = settings.Value;
    _logger = logger;
    _httpClient = httpClient;

    // Set base address once in constructor
    _httpClient.BaseAddress = new Uri(_settings.BaseUrl);

    // Set Authorization header once — all requests reuse it
    _httpClient.DefaultRequestHeaders.Authorization =
        new AuthenticationHeaderValue("Bearer", _settings.ApiKey);

    _httpClient.DefaultRequestHeaders.Accept
        .Add(new MediaTypeWithQualityHeaderValue("application/json"));
}

public async Task<(string PaymentUrl, string InvoiceId)> CreateInvoiceAs
{
    // Build the request object
    var request = new FawaterakInvoiceRequest
    {
        ProviderKey = _settings.ProviderKey,   // ← Critical — required by Faw
        CartTotal = dto.CartTotal,
        Currency = "EGP",
        Customer = new FawaterakCustomer
        {
            FirstName = dto.FirstName,
            LastName = dto.LastName,
            Email = dto.Email,
            Phone = dto.Phone,
            Address = dto.Address
        },
        CartItems = dto.Items.Select(i => new FawaterakCartItem
        {
            Name = i.Name,
            Price = i.Price,
            Quantity = i.Quantity
        }).ToList(),
```

```
            RedirectionUrls = new FawaterakRedirectionUrls
            {
                SuccessUrl = _settings.SuccessUrl,
                FailUrl = _settings.FailUrl,
                PendingUrl = _settings.PendingUrl,
                CallbackUrl = _settings.CallbackUrl
            }
        };

        // Serialize to JSON and log for debugging
        var json = JsonSerializer.Serialize(request, _serializeOptions);
        _logger.LogInformation("Fawaterak request payload: {Json}", json);

        var content = new StringContent(json, Encoding.UTF8, "application/json

        // Make the HTTP call
        var httpResponse = await _httpClient.PostAsync("/api/v2/createInvoiceL
        var body = await httpResponse.Content.ReadAsStringAsync();

        _logger.LogInformation("Fawaterak response ({Status}): {Body}",
            httpResponse.StatusCode, body);

        // First check: HTTP-level failure (4xx, 5xx)
        if (!httpResponse.IsSuccessStatusCode)
            throw new Exception($"Fawaterak HTTP error {httpResponse.StatusC

        // Deserialize response
        var result = JsonSerializer.Deserialize<FawaterakInvoiceResponse>(bod

        // Second check: Fawaterak business-level failure
        if (result?.Status != "success")
            throw new Exception($"Fawaterak returned non-success: {body}");

        return (result.Data.Url, result.Data.InvoiceId.ToString());
    }
}
```

Why two levels of error checking?

Fawaterak can fail in two different ways:

- **HTTP failure** (e.g., 401 Unauthorized, 500 Server Error) — caught by `!httpResponse.IsSuccessStatusCode`
- **Business logic failure** — Fawaterak returns `200 OK` but with `{"status": "error", "message": "..."}` in the body. This is caught by checking `result?.Status != "success"`

Step 7 — Register Services in Program.cs

```
// In Program.cs

// 1. Bind settings from appsettings.json to the strongly-typed class
builder.Services.Configure<FawaterakSettings>(
    builder.Configuration.GetSection("FawaterakSettings"));

// 2. Register HttpClient with typed service
// AddHttpClient<TService> creates a named HttpClient scoped to FawaterakS
builder.Services.AddHttpClient<IFawaterakService, FawaterakService>();
```

Why `AddHttpClient<>` instead of a plain `AddScoped<>`?

`AddHttpClient<IFawaterakService, FawaterakService>()` registers `FawaterakService` in DI while also:

- Managing `HttpClient` lifetime via `IHttpClientFactory` (prevents socket exhaustion)
- Injecting a properly managed `HttpClient` instance into the constructor
- Supporting resilience policies (Polly retry, circuit breaker) if you add them later

---

Step 8 — Create the Payment Controller

```
using ECommerce.Services.Abstraction;
using ECommerce.SharedLibirary.DTO_s.PaymentDTOs;
using Microsoft.AspNetCore.Mvc;
```

```
namespace ECommerce.Presentation.Controllers;

[ApiController]
[Route("api/[controller]")]
public class PaymentController : ControllerBase
{
    private readonly IFawaterakService _fawaterakService;

    public PaymentController(IFawaterakService fawaterakService)
    {
        _fawaterakService = fawaterakService;
    }

    // POST /api/payment/checkout
    [HttpPost("checkout")]
    public async Task<IActionResult> Checkout([FromBody] CreateInvoiceD
    {
        var (paymentUrl, invoiceId) = await _fawaterakService.CreateInvoiceAs

        return Ok(new
        {
            PaymentUrl = paymentUrl,
            InvoiceId = invoiceId
        });
    }

    // GET /api/payment/success
    // Called by Fawaterak when user completes payment successfully
    [HttpGet("success")]
    public IActionResult Success([FromQuery] string? invoiceId)
    {
        // You can update order status in DB here
        return Ok(new { message = "Payment successful", invoiceId });
    }

    // GET /api/payment/fail
    [HttpGet("fail")]
    public IActionResult Fail([FromQuery] string? invoiceId)
```

```
    {
        return BadRequest(new { message = "Payment failed", invoiceId });
    }

    // GET /api/payment/pending
    [HttpGet("pending")]
    public IActionResult Pending([FromQuery] string? invoiceId)
    {
        return Ok(new { message = "Payment pending", invoiceId });
    }

    // POST /api/payment/callback
    // This is the webhook — Fawaterak calls this server-to-server
    [HttpPost("callback")]
    public async Task<IActionResult> Callback([FromBody] FawaterakCallba
    {
        // TODO: Validate HMAC signature for security
        // TODO: Update order status based on payload.PaymentStatus
        // TODO: Mark invoice as paid in your database

        return Ok(); // Must return 200 to acknowledge receipt
    }
}
```

---

5. Configuration & Settings

Environment-specific Configuration

For **production**, never put real API keys in `appsettings.json` (which gets committed to Git). Instead:

```
// appsettings.json — safe to commit, has placeholders
"FawaterakSettings": {
    "ApiKey": "",
    "ProviderKey": "",
    "BaseUrl": "https://staging.fawaterak.com",
    ...
```

```
}

// On the server: use environment variables
// FAWATERAKSETTINGS__APIKEY=your_real_key
// FAWATERAKSETTINGS__PROVIDERKEY=your_real_provider_key
```

ASP.NET Core's configuration system automatically maps `__` (double underscore) in environment variable names to `:` nested keys.

Staging vs Production

Environment	BaseUrl
Staging/Testing	https://staging.fawaterak.com
Production	https://app.fawaterak.com

---

6. The Request/Response Models

What Your App Sends to Fawaterak

```
POST https://staging.fawaterak.com/api/v2/createInvoiceLink
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json

{
  "providerKey": "FAWATERAK.2409",
  "cartTotal": "150.00",
  "currency": "EGP",
  "customer": {
    "first_name": "Ahmed",
    "last_name": "Hassan",
    "email": "ahmed@example.com",
    "phone": "01012345678",
    "address": "Cairo, Egypt"
  },
  "cartItems": [
    {
```

```
      "name": "Product A",
      "price": "50.00",
      "quantity": "3"
    }
  ],
  "redirectionUrls": {
    "successUrl": "https://YOUR_DOMAIN/api/payment/success",
    "failUrl": "https://YOUR_DOMAIN/api/payment/fail",
    "pendingUrl": "https://YOUR_DOMAIN/api/payment/pending",
    "callbackUrl": "https://YOUR_DOMAIN/api/payment/callback"
  }
}
```

What Fawaterak Returns (Success)

```
{
  "status": "success",
  "data": {
    "url": "https://staging.fawaterak.com/pay/invoice/xyz123",
    "invoiceId": 98765
  }
}
```

What Fawaterak Returns (Error)

```
{
  "status": "error",
  "message": "Please try again later!"
}
```

---

7. The Callback (Webhook)

When payment is complete, Fawaterak makes a `POST` request to your `callbackUrl`. The payload looks like:

```
{
  "invoiceId": 98765,
  "paymentStatus": "paid",
  "paymentMethod": "card",
  "amount": "150.00",
  "currency": "EGP"
}
```

The Callback DTO

```
public class FawaterakCallbackPayload
{
    [JsonPropertyName("invoiceId")]
    public long InvoiceId { get; set; }

    [JsonPropertyName("paymentStatus")]
    public string PaymentStatus { get; set; } = default!;

    [JsonPropertyName("paymentMethod")]
    public string PaymentMethod { get; set; } = default!;

    [JsonPropertyName("amount")]
    public string Amount { get; set; } = default!;

    [JsonPropertyName("currency")]
    public string Currency { get; set; } = default!;
}
```

Callback Flow Diagram

```
Fawaterak Server
      │
      │  POST /api/payment/callback
      │  {invoiceId: 98765, paymentStatus: "paid", ...}
      │
      ▼
```

```
Your Callback Endpoint
    │
    ├── Validate HMAC signature (optional but recommended)
    │
    ├── Find Order by InvoiceId in your DB
    │
    ├── Update Order.Status = "Paid"  (if paymentStatus == "paid")
    │   OR Order.Status = "Failed"    (if paymentStatus == "failed")
    │
    └── Return 200 OK  ← IMPORTANT: Fawaterak retries if you don't returr
```

> **Important:** Always return `200 OK` from your callback, even if you encounter a business error. If you return any non-2xx status, Fawaterak will retry the webhook multiple times, potentially processing the same payment notification repeatedly.

---

8. Common Errors & Fixes

Error: `{"status":"error","message":"Please try again later!"}`

This is Fawaterak's generic validation failure. Common causes:

Cause	Symptom	Fix
Missing providerKey in request body	Always fails	Add ProviderKey property to FawaterakInvoiceRequest and assign _settings.ProviderKey
Wrong providerKey value	Always fails	Verify the value matches exactly what's in your Fawaterak dashboard
Expired ngrok URL in callbackUrl	Fails on each run	Restart ngrok, update all 4 URLs in appsettings.json
Invalid/expired ApiKey	Always fails	Regenerate key from Fawaterak dashboard
cartTotal doesn't match sum of cartItems	Sometimes fails	Ensure cartTotal == sum(price * quantity) for all items
Non-string number values	Fails	Make sure price, quantity, cartTotal are all strings like "50.00", not 50.00

Error: Fawaterak HTTP error 401

The ApiKey is invalid or missing from the Authorization header. Check that FawaterakSettings.ApiKey is set correctly in appsettings.json.

Error: System.InvalidOperationException: HttpClient.BaseAddress must be an absolute URI

The BaseUrl in settings is missing the scheme. It must be https://staging.fawaterak.com not staging.fawaterak.com.

Error: Callback never fires locally

Your `callbackUrl` must be publicly reachable. Fawaterak's servers cannot reach `localhost`. Solution:

```
# Start ngrok tunnel
ngrok http 5262

# Copy the https URL shown, e.g.:
# https://abc123.ngrok-free.app

# Update appsettings.json
"CallbackUrl": "https://abc123.ngrok-free.app/api/payment/callback"
```

Error: `NullReferenceException` on `result.Data.Url`

Fawaterak returned `status: "success"` but the `data` field is null or the `url` field is missing. This usually means the response shape changed or the deserialization is wrong. Add logging to inspect the raw response body:

```
_logger.LogInformation("Fawaterak raw body: {Body}", body);
```

---

9. Testing Checklist

Before Testing

- [ ] Fawaterak staging account created at `https://staging.fawaterak.com`
- [ ] `ApiKey` and `ProviderKey` copied from the Fawaterak dashboard
- [ ] ngrok running: `ngrok http YOUR_PORT`
- [ ] All 4 callback URLs updated in `appsettings.json` with the ngrok URL
- [ ] Application restarted after `appsettings.json` changes

Request Validation

- [ ] `cartTotal` is a string in format `"150.00"`
- [ ] Each cart item `price` and `quantity` are strings
- [ ] Customer `phone` is a valid Egyptian number (e.g., `"01012345678"`)
- [ ] Customer `email` is a valid email address
- [ ] `providerKey` is present and matches dashboard value

Testing the Flow

1. Start your API
2. Use Postman or Swagger to call `POST /api/payment/checkout` with a sample body:

```
{
  "cartTotal": "100.00",
  "firstName": "Test",
  "lastName": "User",
  "email": "test@example.com",
  "phone": "01012345678",
  "address": "Cairo, Egypt",
  "items": [
    {
      "name": "Test Product",
      "price": "100.00",
      "quantity": "1"
    }
  ]
}
```

3. Expect a response with `paymentUrl` and `invoiceId`
4. Open `paymentUrl` in the browser
5. Use Fawaterak test card: `4111 1111 1111 1111`, any future expiry, any CVV
6. Check your ngrok console — you should see the callback request hit your server
7. Verify you're redirected to your `successUrl`

---

10. Production Readiness Checklist

Item	Action
Swap `BaseUrl` to production	`https://app.fawaterak.com`
Move secrets to environment variables	Remove ApiKey from appsettings.json
Implement HMAC validation on callback	Prevents fake webhook calls
Persist invoiceId with each order	Needed to match callback to order
Implement idempotency in callback handler	Prevent duplicate order updates on retry
Add Polly retry policy to HttpClient	Handle transient Fawaterak failures gracefully
Set up logging/monitoring	Log all Fawaterak request/response payloads
Replace ngrok with real domain	Use your actual deployed API URL

Guide generated for the ECommerce Web API Revision project — github.com/sefffo/Web-API-Revesion